MyAccounting System — Architecture Documentation

Version: 1.0.0
Release Date: 2026-04-12
Author: Daniel Seguin
Description: Business accounting software for the service industry.

Table of Contents

1. Overview
2. Technology Stack
3. High-Level Architecture
4. Application Layers
   - 4.1 Presentation Layer
   - 4.2 Business Logic Layer
   - 4.3 Data Access Layer
   - 4.4 Database Layer
5. Module Descriptions
6. Authentication & Security
7. Internationalization (i18n)
8. Reporting & Export
9. Email Integration
10. Google Services Integration
11. Password Manager
12. Deployment & Distribution
13. Module Dependency Diagram
14. File Structure

---

1. Overview

MyAccounting is a desktop accounting application built with Python and Tkinter that targets small to medium businesses in the service industry. It provides a full double-entry general ledger, accounts receivable invoicing, cash receipts, timesheet management, and financial reporting, all operating against a Microsoft SQL Server database.

The system is designed to run on Windows desktops, supports English and French interfaces, and can be packaged as a standalone executable using PyInstaller.

---

2. Technology Stack

Component	Technology
Language	Python 3.x
GUI Framework	Tkinter + ttk (standard library)
Database	Microsoft SQL Server (local instance)
Database Driver	pyodbc + ODBC Driver 17 for SQL Server
PDF Generation	ReportLab
Excel Export/Import	openpyxl
Internationalization	Python `gettext` (.po / .mo files)
Password Encryption	`cryptography` library (Fernet symmetric encryption)
Calendar Widget	tkcalendar
Embedded Browser	tkinterweb (optional, for Google services)
Packaging	PyInstaller
Connection Mode	Windows Trusted Connection (no username/password)

---

3. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                       MyAccounting Application                   │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │                      app.py  (Main Window)               │    │
│  │   Sidebar navigation · Dashboard · To-do widget          │    │
│  └──────────────────────┬───────────────────────────────────┘    │
│                         │ loads modules into content pane        │
│  ┌──────────────────────▼───────────────────────────────────┐    │
│  │                    Functional Modules                    │    │
│  │  GL  |  Journal  |  Reports  |  Config  |  Invoice  |  ...  |    │
│  └──────────────────────┬───────────────────────────────────┘    │
│                         │ calls                                  │
│  ┌──────────────────────▼───────────────────────────────────┐    │
│  │             Data Access Layer (db.py)                    │    │
│  │   get_connection() · ensure_app_tables() · helpers       │    │
│  └──────────────────────┬───────────────────────────────────┘    │
│                         │ pyodbc                                 │
└─────────────────────────┼────────────────────────────────────────┘
                          │
              ┌───────────▼──────────────┐
              │  Microsoft SQL Server    │
              │  Database: myaccounting  │
              │  Trusted_Connection=yes  │
              └──────────────────────────┘
```

---

4. Application Layers

4.1 Presentation Layer

Entry point: `app.py`

The GUI is built entirely with Python's `tkinter` library and the `ttk` themed widget set. The main window follows a sidebar-navigation + content-pane layout:

- **Sidebar** (left, fixed 200 px): dark-themed navigation buttons, one per functional module. Hover and active states provide visual feedback.
- **Content pane** (right): replaced with the selected module's `ttk.Frame` when the user clicks a navigation button.
- **Dashboard**: displayed on startup showing quick KPI cards (AR Balance, Revenue MTD, Cash Balance, Draft Invoices, Overdue Invoices, Open Timesheets) and a mini to-do list widget.

Color palette:

Constant	Hex	Usage
`NAV_BG`	`#2c3e50`	Sidebar background
`NAV_BTN_BG`	`#34495e`	Nav button normal
`NAV_ACTIVE`	`#2980b9`	Active nav button
`_ACCENT`	`#2980b9`	Accent color
`_GREEN`	`#27ae60`	Positive indicators
`_RED`	`#e74c3c`	Negative / alert

4.2 Business Logic Layer

Each functional module is implemented as a self-contained Python class that inherits from `ttk.Frame`. Modules own their own Treeview lists, dialog sub-windows, and business rules (validation, calculations, posting logic). Common utilities are shared via `utils.py`.

Key business rules implemented at the module level:

- **Double-entry validation**: journal entries must balance (total debits = total credits) before posting.
- **Invoice tax calculation**: applied using the configured tax code and its component rates.
- **Cash receipt posting**: automatically generates a GL debit to the bank account and a credit to Accounts Receivable.
- **Timesheet totalling**: daily hours are summed from work-item lines; the parent detail row stores the aggregate.

4.3 Data Access Layer

File: `db.py`

All database interaction is centralized in `db.py`. It provides:

- `get_connection()` — returns a new `pyodbc` connection using Windows Trusted Authentication.
- `ensure_app_tables()` — idempotent schema initialization. Called once at startup via `app.py`; creates all tables if missing and runs safe ALTER TABLE migrations for schema evolution.
- `get_settings(conn)` / `save_setting(conn, key, value)` — key-value settings helpers.
- Helper functions for invoice attachments, GL posting, and timesheet operations.

The data access strategy is a **thin repository pattern**: modules open a connection, do their work (often a single query or transaction), then close the connection immediately. Long-lived connections are avoided.

4.4 Database Layer

Server: `localhost`
Database: `myaccounting`
Authentication: Windows Trusted Connection (SSPI)
Driver: ODBC Driver 17 for SQL Server

The database schema is managed code-first through `db.py:ensure_app_tables()`. There are no external migration tools; all schema changes are handled with `IF NOT EXISTS` guards and `ALTER TABLE` statements executed at application startup.

---

5. Module Descriptions

5.1 `app.py` — Main Application Window

Responsibility	Details
Application bootstrap	Reads `app.json`, calls `db.ensure_app_tables()`, initializes `i18n`, shows `LoginDialog`
Window layout	Creates sidebar + content pane; instantiates and caches all module frames
Navigation	Raises the appropriate module frame on sidebar button click
Dashboard	Displays 6 KPI cards and a scrollable to-do widget
About dialog	Reads and displays `app.json` metadata

5.2 `gl_module.py` — General Ledger / Chart of Accounts

Provides full CRUD management of GL accounts (`glmaster`) and read-only browsing of transaction history (`gldetails`).

Feature	Details
Account list	Sortable `Treeview` with account number, name, balance type, opening balance
Add / Edit / Delete	Modal `AccountDialog`
Transaction history	Filtered by date range; shows source document number
Export	Excel (.xlsx), CSV, PDF via ReportLab

5.3 `journal_module.py` — General Journal

Manages manual double-entry journal entries through a two-level master/detail structure.

Feature	Details
Entry list	Filtered by status (Draft / Posted)
Line items	`LineDialog` per debit/credit line
Balance check	Entry cannot be posted unless debits = credits
Posting	Transfers journal lines to `gldetails`; status changes to Posted
Export	CSV, PDF verification listing

5.4 `reports_module.py` — Financial Reports

Generates standard accounting reports computed live from the database.

Report	Description
Trial Balance	All accounts with debit/credit net balance
Income Statement	Revenue & expense accounts (4000–9999) for a date range
Balance Sheet	Asset, liability, equity accounts (1000–3999) as of a date

Export formats: PDF (ReportLab), Excel (openpyxl).

5.5 `config_module.py` — Configuration

Multi-tab settings panel covering all application administration.

Tab	Contents
Company Info	Name, address, phone, email, website, logo
Fiscal Year	Fiscal dates, default currency, tax rate, AR / Revenue / Tax / Cash GL accounts
Customers	Full CRUD, including billing/shipping addresses, salesman, tax code, credit limit, payment terms
Inventory	Full CRUD for products and services
Projects	Project codes and start dates
Salesmen	Consultant profiles with Google OAuth credentials and signature image
Tax Codes	Tax code + multi-component rate definitions
Payment Terms	Named payment term descriptions
Users	User accounts with roles (admin / accountant)
User Access	Per-module access levels (full / read-only / none)
Email Settings	SMTP host/port/credentials for invoice and timesheet emails

5.6 `invoice_module.py` — Invoicing

Full accounts-receivable invoicing with PDF generation.

Feature	Details
Invoice list	Filtered by status, customer, date range
Line items	Items from inventory; manual line items; header/comment lines
Tax	Multi-component tax via `taxcodedetail`
PDF generation	Professional layout with company logo, salesman signature
Email	Sends PDF via SMTP directly from invoice
Attachments	File attachments stored as file-system paths in `invoice_attachments`
GL posting	Posts AR debit + Revenue credit + Tax credit to `gldetails`

5.7 `cash_receipt_module.py` — Cash Receipts

Records customer payments and posts them to the GL.

Feature	Details
Receipt entry	Customer, date, amount, linked invoice, bank account
Verification	Pre-post listing showing proposed GL entries
GL posting	DR Bank account, CR Accounts Receivable

5.8 `timesheet_module.py` — Timesheet Management

Tracks consultant work hours by month with multi-work-item daily granularity.

Feature	Details
Timesheet list	Filtered by year, month, consultant
Daily grid	AM / PM / Other time slots; N work-item lines per day
Hours calculation	Work-item hours roll up to day total
PDF export	Professional landscape timesheet with company logo and signature
Excel export	openpyxl workbook
Email	Sends PDF timesheet to client manager via SMTP or Google OAuth
CSV import	Imports hours from external timesheet CSV files

5.9 `utilities_module.py` — Google Services & Utilities

Provides quick-launch tiles for external Google services and access to the Password Manager.

Tile	Target
Gmail	https://mail.google.com
Google Calendar	https://calendar.google.com
Google Drive	https://drive.google.com
Password Manager	Opens the internal `PasswordManagerModule`

When `tkinterweb` is installed the services can open in an embedded browser pane; otherwise they open in the system default browser.

5.10 `password_manager.py` — Password Manager

Encrypted credential vault stored in SQL Server.

- Passwords are encrypted with **Fernet symmetric encryption** before being written to the database.
- The Fernet key is stored in `pm.key` in the application directory.
- Each entry is scoped to the logged-in user.

5.11 `auth.py` — Authentication

Function	Description
`hash_password(plain)`	PBKDF2-HMAC-SHA256, 260 000 iterations, random 16-byte salt
`verify_password(plain, stored)`	Constant-time comparison via `hmac.compare_digest`
`authenticate_user(username, password)`	Returns user dict or `None`
`get_user_access(user_id, role)`	Returns per-module access dict; admins always get `"full"`

5.12 `login_dialog.py` — Login Dialog

Modal Toplevel window presented before the main window becomes visible. Prevents unauthorized access; passes the authenticated user dict to `app.py`.

5.13 `i18n.py` — Internationalization

Wraps Python `gettext` to provide runtime language switching.

- Supported languages: **English** (`en`) and **French** (`fr`).
- Translation catalogues: `locale/<lang>/LC_MESSAGES/myaccounting.mo`.
- Single public function `_()` used everywhere in place of string literals.
- Language preference is stored in the `settings` table (`ui_language` key).

5.14 `utils.py` — Shared Utilities

Stateless helper functions used across all modules.

Function	Description
`fmt_currency(value)`	Decimal formatting with thousands separator
`fmt_date(value)`	Converts date/string to MM/DD/YYYY
`parse_date(value)`	Parses user date string to `datetime.date`
`parse_amount(value)`	Parses currency string to `float`
`today_str()`	Returns today as MM/DD/YYYY
`make_date_field(parent, ...)`	Builds a date Entry + calendar-picker button

---

6. Authentication & Security

Login Flow

```
app.py startup
    └➤ LoginDialog shown (blocks main window)
            └➤ auth.authenticate_user(username, password)
                    └➤ db.get_connection() → query users table
                    └➤ verify_password() with PBKDF2 + hmac.compare_digest
            └➤ On success: user dict passed to App
               On failure: error message shown; limited retry attempts
```

Role Model

Role	Description
`admin`	Full access to all modules; bypasses `useraccess` table
`accountant`	Access controlled per-module by `useraccess` table

Access Levels

Level	Description
`full`	Can read and write
`view only`	Read-only; editing controls are disabled
(no row)	Defaults to `view only`

Password Security

- Algorithm: **PBKDF2-HMAC-SHA256**
- Iterations: **260 000**
- Salt: 16 random bytes per password
- Stored format: `<salt_hex>:<hash_hex>` in `NVARCHAR(255)`
- Comparison: `hmac.compare_digest` (constant-time, prevents timing attacks)

---

7. Internationalization (i18n)

```
i18n.setup(lang)          ← called once in app.py with settings['ui_language']
    ├─ lang='en'  → NullTranslations (pass-through)
    └─ lang='fr'  → gettext.translation('myaccounting', locale/fr/LC_MESSAGES/)

All UI strings: _("Some text")   ← macro resolves at runtime
```

Translation files are compiled with `compile_mo.py` from `.po` source files. Both `.po` (source) and `.mo` (compiled binary) are shipped. PyInstaller bundles the `locale/` directory at build time.

8. Reporting & Export

PDF (ReportLab)

Used by: GL export, Journal verification listing, Reports (Trial Balance, Income Statement, Balance Sheet), Invoice PDF, Timesheet PDF.

All PDF generation uses `reportlab.platypus` (`SimpleDocTemplate`, `Table`, `Paragraph`, `Spacer`, `HRFlowable`, `Image`). Company logo is embedded when configured.

Excel (openpyxl)

Used by: GL export, Reports export, Timesheet export.

Workbooks use styling (`Font`, `Alignment`, `PatternFill`, `Border`) to produce professional-looking output.

CSV

Used by: GL export, Journal CSV export, Timesheet CSV import.

---

9. Email Integration

SMTP email is used in two modules:

Module	Purpose
`invoice_module.py`	Send PDF invoice to customer
`timesheet_module.py`	Send PDF timesheet to client manager

Configuration is stored in the `settings` table:

Key	Description
`smtp_host`	SMTP server hostname
`smtp_port`	SMTP port (default 587)
`smtp_user`	SMTP login username
`smtp_password`	SMTP login password
`smtp_tls`	`1` = use STARTTLS
`invoice_email_subject`	Subject template with `{invoice_number}`, `{company}` placeholders
`invoice_email_body`	Body template with `{customer}`, `{total}`, `{consultant}`, etc. placeholders

Timesheets also support sending via **Google OAuth** when a salesman has a configured `google_refresh_token`.

---

10. Google Services Integration

Salesmen can configure Google OAuth 2.0 credentials in their salesman profile (`Configuration → Salesmen → Edit`). When configured:

- OAuth tokens are stored per salesman in the `salesmen` table (encrypted-at-rest at the transport layer via TLS; stored in SQL Server with application-level access controls).
- Timesheets can be emailed directly using the salesman's Gmail identity.

Google service tiles (Gmail, Calendar, Drive) open in an embedded `tkinterweb` browser pane when available, or in the system browser as a fallback.

---

11. Password Manager

The Password Manager stores application credentials (e.g., external system passwords) in SQL Server, encrypted with **Fernet symmetric encryption**.

```
pm.key  (file, application directory)
    └→ Fernet key (generated on first use, must be backed up)
            └→ encrypt_password(plain)  →  token stored in passwords.password_enc
            └→ decrypt_password(token)  →  plain shown in UI only on request
```

Each entry is owned by a `user_id` (FK to `users`). Entries are only visible to their owner.

---

12. Deployment & Distribution

Development Run

```
python app.py
```

Requirements: `pip install pyodbc reportlab openpyxl tkcalendar cryptography`

Production Build (PyInstaller)

```
build.bat  →  pyinstaller MyAccounting.spec
```

Output: `dist/MyAccounting/MyAccounting.exe` (directory mode, not single-file).

The `.spec` file configures:

- Hidden imports for `pyodbc`, `reportlab`, `openpyxl`
- Data folder inclusions: `locale/`, `app.json`
- Runtime hook: `rthook_tcl.py` (ensures Tcl/Tk paths are correct)

Deployment Package

`DeploymentPackage/` contains `deploy.py` and `deploy.spec` for creating a database setup / installer executable that creates the SQL Server database and initial schema.

---

13. Module Dependency Diagram

```
app.py
 ├── auth.py
 │    └── db.py
 ├── i18n.py
 ├── login_dialog.py
 │    └── auth.py
 ├── db.py
 ├── utils.py
 ├── gl_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 ├── journal_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 ├── reports_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 ├── config_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 ├── invoice_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 ├── cash_receipt_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 ├── timesheet_module.py
 │    ├── db.py
 │    ├── i18n.py
 │    └── utils.py
 └── utilities_module.py
      ├── db.py
      ├── i18n.py
      └── password_manager.py
           └── db.py

External libraries: pyodbc · reportlab · openpyxl · tkcalendar · cryptography · tkinterweb
```

14. File Structure

```
MyAccounting/
│
├── app.py                    # Main entry point and application window
├── app.json                  # Application metadata (name, version, author)
├── auth.py                   # Authentication and password hashing
├── db.py                     # Database connection and schema management
├── i18n.py                   # Internationalization (gettext wrapper)
├── utils.py                  # Shared formatting and utility functions
├── login_dialog.py           # Login dialog window
│
├── gl_module.py              # General Ledger / Chart of Accounts module
├── journal_module.py         # General Journal entry module
├── reports_module.py         # Financial reports module
├── config_module.py          # Configuration / administration module
├── invoice_module.py         # Invoicing module
├── cash_receipt_module.py    # Cash receipts module
├── timesheet_module.py       # Timesheet management module
├── utilities_module.py       # Google services & utilities module
├── password_manager.py       # Encrypted password vault module
│
├── import_data.py            # Data import utility (CSV / text files)
├── compile_mo.py             # Compiles .po translation files to .mo
│
├── MyAccounting.spec         # PyInstaller build specification
├── build.bat                 # Windows build script
├── rthook_tcl.py             # PyInstaller Tcl/Tk runtime hook
│
├── pm.key                    # Fernet encryption key (generated at runtime)
│
├── locale/
│   ├── en/LC_MESSAGES/
│   │   ├── myaccounting.po   # English translation source
│   │   └── myaccounting.mo   # English translation compiled
│   └── fr/LC_MESSAGES/
│       ├── myaccounting.po   # French translation source
│       └── myaccounting.mo   # French translation compiled
│
├── testcases/                # Unit and integration test files
├── samples/                  # Sample CSV files for testing
├── importdir/                # Default import directory
├── exportdir/                # Default export directory
├── documentation/            # Project documentation
└── DeploymentPackage/        # Installer / deployment scripts
```